Trabajo Practico 1: Metodos de Busqueda

Sistemas de Inteligencia Artificial

ITBA – 2026

Índice

1. Introduccion **3**

2. Parte 1: Heuristicas para el 8-puzzle **3**
2.1. Planteo del problema 3
2.2. Estructura de estado 3
2.3. Observacion clave: la ficha 5 va al centro 3
2.4. Heuristica 1: h_5 – distancia del 5 al centro 3
2.5. Heuristica 2: h_G – Manhattan minima sobre todos los goals 4
2.6. Combinacion 4
2.7. Metodos de busqueda recomendados 4

3. Parte 2: Sokoban **5**
3.1. Descripcion del problema 5
3.2. Representacion del estado 5

4. Algoritmos de busqueda implementados **5**

5. Heuristicas implementadas **5**
5.1. Heuristica 1: Deadlocks estaticos (h_{deadlock}) 5
5.2. Heuristica 2: Minimum matching (h_{match}) 6
5.3. Heuristica combinada (h_{combined}) 6

6. Resultados experimentales **6**
6.1. Costo por nivel 7
6.2. Algoritmos optimos: tiempo, nodos y frontera 8
6.3. Algoritmos no optimos: DFS y Greedy 9
6.4. Comparativas directas 10
6.5. Distribucion de costo 11

7. Analisis **11**
7.1. Optimalidad 11
7.2. Eficiencia de las heuristicas 12
7.3. Rol de static_deadlock 12
7.4. Determinismo de las iteraciones 12

8. Metricas reportadas **12**

9. Tests **13**

10. Conclusiones **13**

1. Introduccion

Este trabajo practico aborda el estudio de metodos de busqueda en inteligencia artificial a traves de dos ejercicios. El primero es un analisis teorico de heuristicas para una variante del **8-puzzle con multiples estados objetivo**. El segundo es la implementacion completa de un solver de **Sokoban** usando los algoritmos BFS, DFS, Greedy y A*, con al menos dos heuristicas admisibles.

El codigo fuente, instrucciones de ejecucion y niveles de prueba se encuentran en el repositorio del proyecto. Para ejecutar el solver:

```
python3 -m src.main --iterations 10 --seed 42
```

2. Parte 1: Heuristicas para el 8-puzzle

2.1. Planteo del problema

Se trabaja con una variante del 8-puzzle en la que la condicion de solucion no es una disposicion fija, sino que las fichas deben cumplir **restricciones de adyacencia**. Cada ficha numerada del 1 al 8 debe ser vecina (horizontal o vertical) de ciertos numeros especificos. El grafo de adyacencia requerido es:

Ficha	Adyacentes requeridos
1	2, 4
2	1, 3, 5
3	2, 6
4	1, 5, 7
5	2, 4, 6, 8
6	3, 5, 9
7	4, 8
8	5, 7, 9

2.2. Estructura de estado

Un estado se define por las posiciones de las 9 fichas (1–8 mas el vacio) en el tablero 3×3. Se puede representar como un arreglo 1D de 9 elementos. Las acciones posibles son mover el espacio vacio a una casilla adyacente (arriba, abajo, izquierda, derecha), respetando los limites del tablero.

2.3. Observacion clave: la ficha 5 va al centro

En el grafo de adyacencias, la ficha **5** tiene grado 4 (es adyacente a 2, 4, 6 y 8). En un tablero 3×3, la unica casilla con 4 vecinos es el **centro**. Por lo tanto, en toda solucion valida, el 5 debe estar en la casilla central. Esto genera multiples estados meta posibles (rotaciones y reflexiones). Llamamos G al conjunto de todos los estados solucion.

2.4. Heuristica 1: h_5 – distancia del 5 al centro

Definimos una heuristica por casos basada en la posicion del 5 y del vacio:

$$h_5(s) = \begin{cases} d_M(\square, 5) - 1 & \text{si 5 en esquina y vacio no adyacente a 5} \\ d_M(5, c) & \text{si 5 en esquina y vacio adyacente a 5} \\ d_M(\square, c) & \text{si 5 en borde y vacio no en centro} \\ d_M(5, c) & \text{si 5 en borde y vacio en centro} \\ 0 & \text{si 5 ya en centro} \end{cases}$$

donde d_M denota distancia Manhattan, $\square$ es el vacio y c es la casilla central.

Admisibilidad: En cada caso, h_5 mide una condicion necesaria antes de alcanzar cualquier solucion. Caso 1: antes de mover al 5, el vacio debe llegar adyacente a el ($\geq k-1$ movimientos). Caso 2: la distancia Manhattan del 5 al centro requiere al menos esa cantidad de movimientos del 5. Caso 3: el vacio debe pasar por el centro. Caso 4: falta al menos 1 movimiento del 5. Caso 5: ya cumplido. En todos los casos $h_5(s) \leq h^*(s)$.

2.5. Heuristica 2: h_G – Manhattan minima sobre todos los goals

$$h_G(s) = \min_{g \in G} \sum_{i=1}^{8} d_M(\text{pos}_s(i), \text{pos}_g(i))$$

donde G es el conjunto de estados solucion validos y la suma recorre fichas 1 a 8 (excluyendo el vacio).

Admisibilidad: Para un goal fijo g, la suma Manhattan es admisible: cada movimiento desplaza una sola ficha una casilla, asi que la suma decrece a lo sumo en 1 por jugada. Como esto vale para cada $g \in G$, el minimo tambien es admisible.

2.6. Combinacion

$$h(s) = \max\big(h_5(s),\ h_G(s)\big)$$

El maximo de dos heuristicas admisibles es admisible: $\max(h_1, h_2) \leq \max(h^*, h^*) = h^*$. Se usa máx y no suma porque la suma podria doble-contar esfuerzo, rompiendo admisibilidad. La combinacion aprovecha la informacion estructural de h_5 (centrar el 5) y la informacion global de h_G (ordenar todas las fichas).

2.7. Metodos de busqueda recomendados

- **A*** con $h = \max(h_5, h_G)$: optimo (heuristicas admisibles) y eficiente.
- **Greedy** con h: rapido pero sin garantia de optimalidad.
- **BFS**: optimo con costos unitarios pero explora mas estados.
- **DFS**: encuentra alguna solucion, sin garantia de optimalidad.

3. Parte 2: Sokoban

3.1. Descripcion del problema

Sokoban es un puzzle donde un jugador empuja cajas hasta posiciones objetivo dentro de un tablero con paredes. Las reglas: el jugador se mueve en 4 direcciones, las cajas solo se empujan (no se tiran), solo una caja a la vez, y el puzzle se resuelve cuando todas las cajas estan sobre todos los goals.

3.2. Representacion del estado

Cada estado se modela con la clase `SokobanState`:

- **Posicion del jugador**: tupla $(fila, columna)$.
- **Posiciones de las cajas**: `frozenset` de posiciones (inmutable y hasheable).
- **Layout estatico**: `BoardLayout` compartido entre estados (paredes, goals, piso).

La condicion de goal es: `boxes` = `goals` (igualdad de conjuntos).

Cada accion (movimiento del jugador) tiene costo 1, independientemente de si empuja una caja o no. La identidad de un estado para el conjunto de explorados esta dada por $(jugador, cajas)$.

4. Algoritmos de busqueda implementados

Algoritmo	Tipo	Estructura	Optimo	Completo
BFS	Desinformado	Cola FIFO	Si	Si
DFS	Desinformado	Pila LIFO	No	No
Greedy	Informado	Min-heap por h	No	No
A*	Informado	Min-heap por $g + h$	Si (h admisible)	Si

Detalle de implementacion de A*: La implementacion usa graph search con reapertura de nodos cerrados. Cuando A* encuentra un camino mas barato a un estado ya expandido, lo reabre y re-expande. Esto garantiza optimalidad con heuristicas admisibles (no requiere consistencia). Se usa un diccionario `best_g` para trackear el mejor costo conocido por estado, y `closed_g` para los expandidos. Si un sucesor tiene menor costo que `best_g`, se actualiza y se reinserta en el heap.

El tie-breaking en el heap usa la tupla $(f, h, -g, secuencia)$: ante igual f, prefiere menor h (mas cercano al goal); ante igual h, prefiere mayor g (mas profundo).

5. Heuristicas implementadas

5.1. Heuristica 1: Deadlocks estaticos (h_{deadlock})

Detecta posiciones donde una caja **nunca puede llegar a ningun goal**, usando un modelo relajado de una sola caja sin obstaculos.

Algoritmo: BFS inverso desde los goals. Para cada posicion, se evalua desde donde podria haber llegado una caja con un empuje valido (la posicion previa de la caja y la posicion de soporte del jugador deben ser piso). Las posiciones no alcanzadas son **prohibidas**.

$$h_{\text{deadlock}}(s) = \begin{cases} +\infty & \text{si alguna caja (fuera de goal) esta en posicion prohibida} \\ 0 & \text{caso contrario} \end{cases}$$

Admisibilidad: Si devuelve ∞, el estado es irrecuperable (costo real tambien infinito). Si devuelve 0, nunca sobreestima. El analisis se precomputa una vez por layout y se cachea con `@lru_cache`.

5.2. Heuristica 2: Minimum matching (h_{match})

Calcula la asignacion optima de cajas no colocadas a goals no ocupados, minimizando la suma de distancias Manhattan.

$$h_{\text{match}}(s) = \min_{\sigma} \sum_{i} d_M(\text{caja}_i, \text{goal}_{\sigma(i)})$$

donde las cajas son $B = \texttt{boxes} - \texttt{goals}$ (no colocadas) y los goals son $G' = \texttt{goals} - \texttt{boxes}$ (no ocupados).

Se resuelve con el **algoritmo Hungaro** via `scipy.optimize.linear_sum_assignment` en $O(n^3)$. Optimizaciones: shortcircuit para 0 cajas (retorna 0) y para 1 caja (retorna Manhattan directo sin numpy).

Admisibilidad: Manhattan es lower bound del costo real de empujar una caja (ignora paredes y otras cajas). El matching minimo no doble-cuenta porque asigna cada goal a una sola caja.

5.3. Heuristica combinada (h_{combined})

$$h_{\text{combined}}(s) = \max\big(h_{\text{match}}(s),\ h_{\text{deadlock}}(s)\big)$$

Admisibilidad: El maximo de dos heuristicas admisibles es admisible. En la practica, h_{deadlock} aporta poda de estados imposibles y h_{match} aporta guia numerica.

6. Resultados experimentales

Se ejecutaron los 8 metodos (BFS, DFS, Greedy×3 heuristicas, A*×3 heuristicas) sobre 4 niveles con 10 iteraciones cada uno (seed=42). Las metricas reportadas son: resultado (exito/fracaso), costo de la solucion, nodos expandidos, nodos en frontera al finalizar, y tiempo de procesamiento.

Todos los metodos resolvieron todos los niveles (success_rate = 1,0).

6.1. Costo por nivel

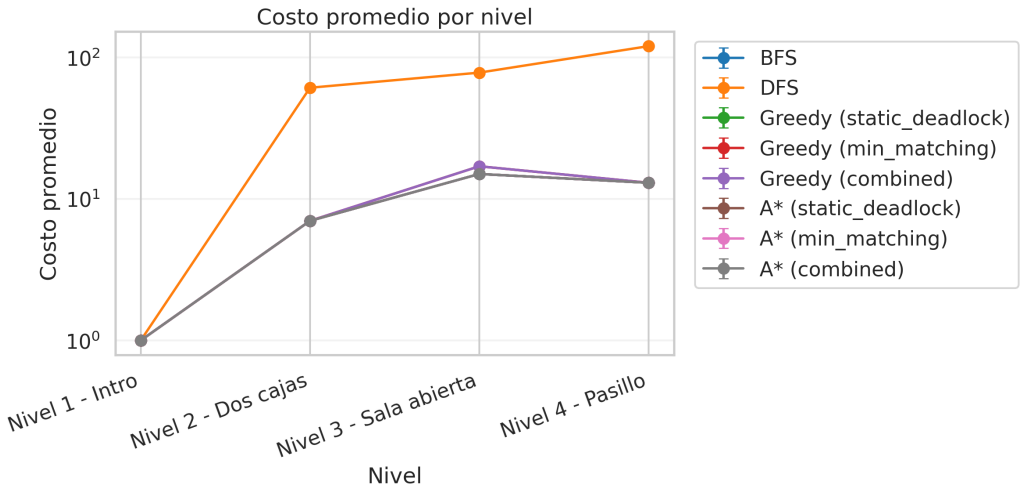

Figura 1: Costo promedio por nivel. BFS y A* (todos los variantes) encuentran la solucion optima. DFS encuentra soluciones significativamente mas largas. Greedy con min_matching/combined empata el optimo en estos niveles.

Metodo	**Nivel 1**	**Nivel 2**	**Nivel 3**	**Nivel 4**
BFS	1	7	15	13
DFS	1	61	78	120
Greedy (min_matching)	1	7	17	13
Greedy (static_deadlock)	1	7	15	13
A* (combined)	1	7	15	13
A* (min_matching)	1	7	15	13
A* (static_deadlock)	1	7	15	13

DFS encuentra soluciones de costo 61, 78 y 120 donde el optimo es 7, 15 y 13 respectivamente. Esto demuestra que DFS no garantiza optimalidad.

6.2. Algoritmos optimos: tiempo, nodos y frontera

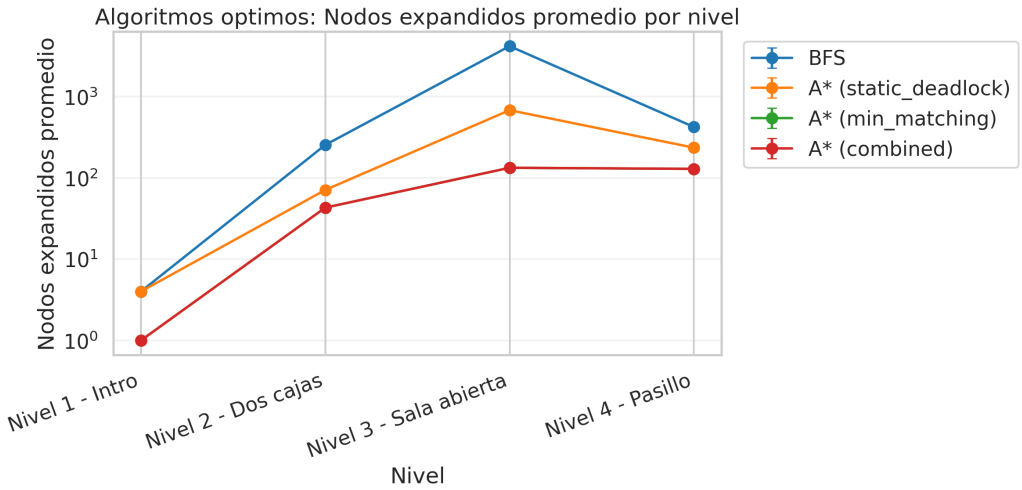

Figura 2: Nodos expandidos por los algoritmos optimos. BFS explora significativamente mas que A*. A*(combined) y A*(min_matching) coinciden porque min_matching es consistente y no requiere reapertura.

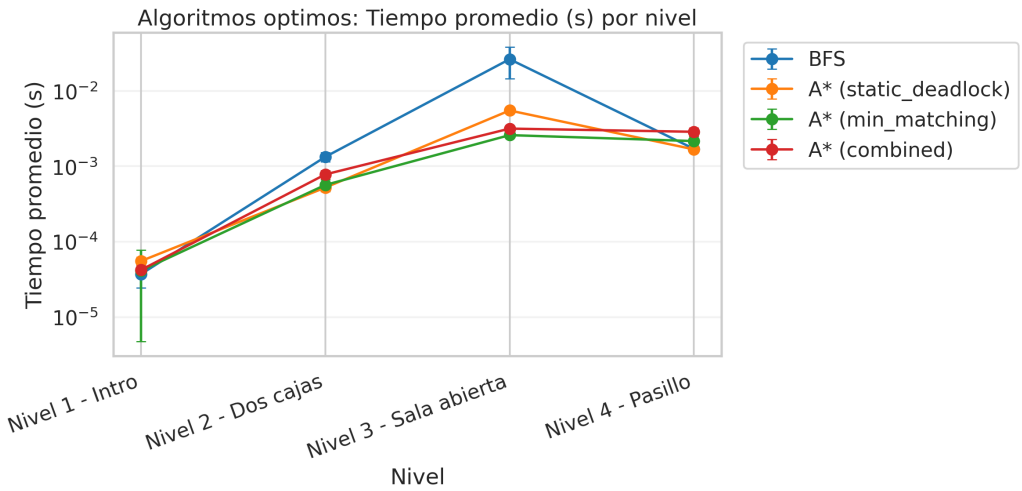

Figura 3: Tiempo de ejecucion de los algoritmos optimos. Escala logaritmica. A* reduce el tiempo respecto de BFS en los niveles con mayor branching (Nivel 3).

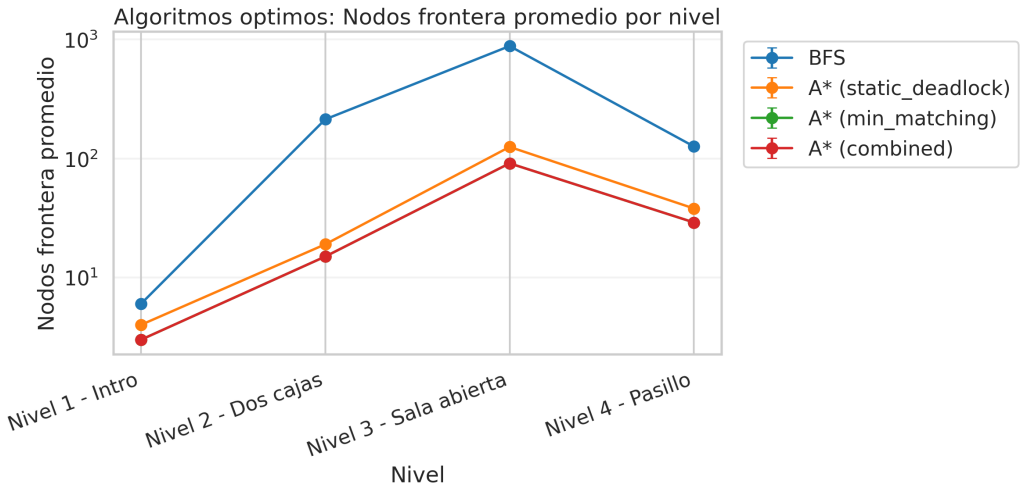

Figura 4: Nodos en frontera al finalizar. A*(combined) mantiene la frontera mas chica. Nota: esta metrica es la frontera al terminar, no el pico de memoria.

Nodos expandidos	**Nivel 1**	**Nivel 2**	**Nivel 3**	**Nivel 4**
BFS	4	254	4171	422
A* (static_deadlock)	4	71	680	235
A* (min_matching)	1	43	133	129
A* (combined)	1	43	133	129

A*(combined) expande 31× menos nodos que BFS en Nivel 3 (133 vs 4171). A*(static_deadlock) queda intermedio porque $h = 0$ para la mayoria de estados (no guia la busqueda, solo poda deadlocks).

6.3. Algoritmos no optimos: DFS y Greedy

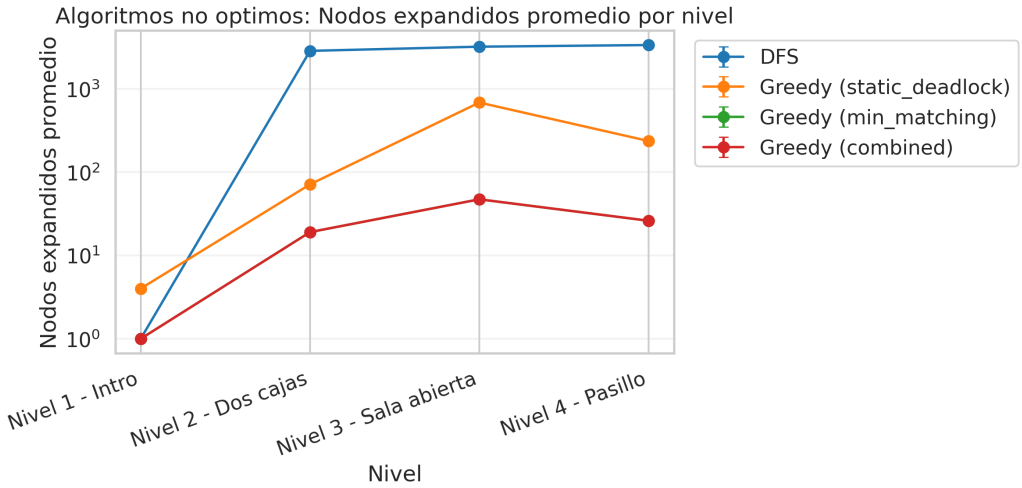

Figura 5: Nodos expandidos por DFS y las variantes de Greedy. DFS domina con miles de nodos. Greedy con min_matching o combined expande menos de 50 nodos.

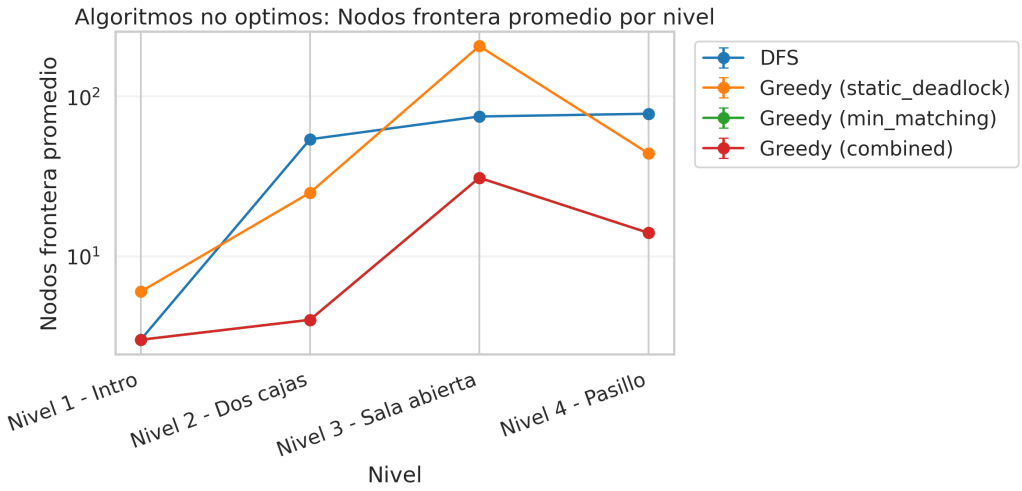

Figura 6: Frontera de los algoritmos no optimos. Greedy(static_deadlock) acumula frontera comparable a DFS porque $h = 0$ no guia la busqueda.

6.4. Comparativas directas

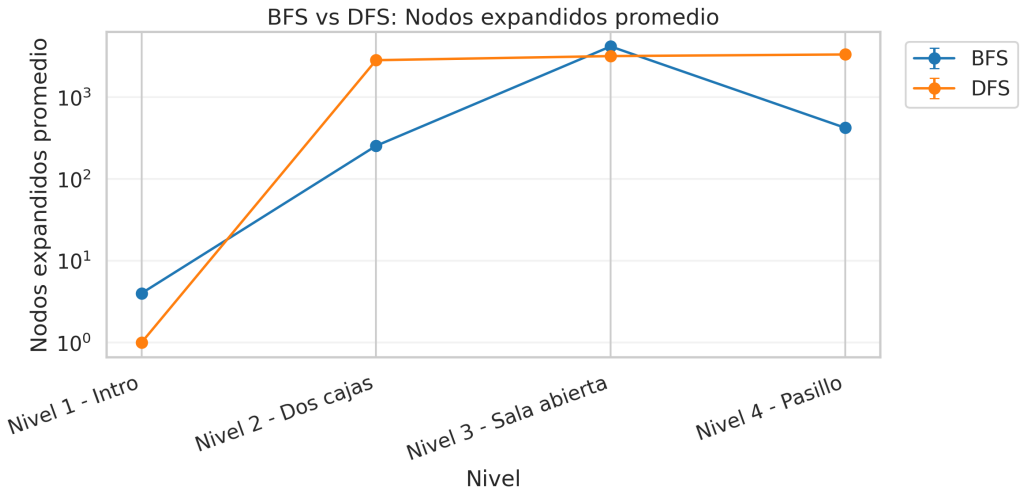

Figura 7: BFS vs DFS: nodos expandidos. DFS expande mas nodos en todos los niveles excepto Nivel 3 donde se acercan.

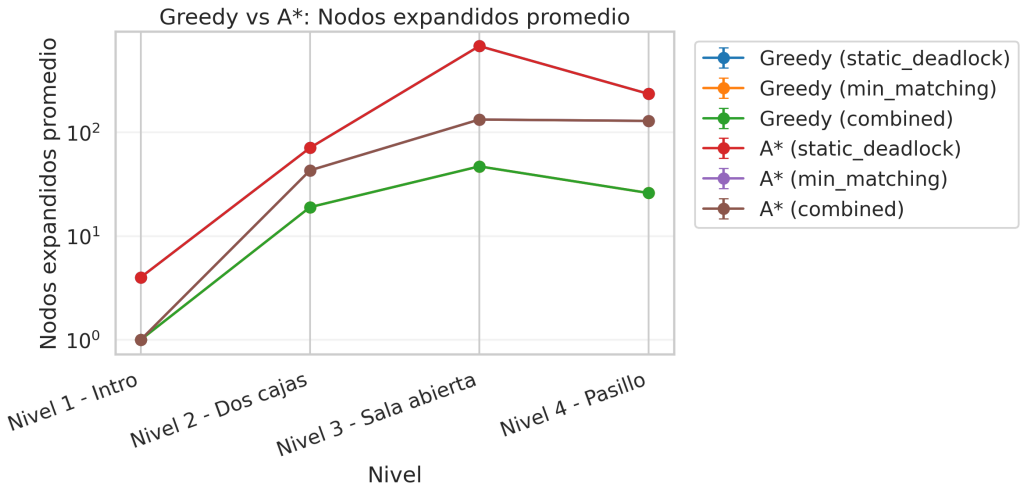

Figura 8: Greedy vs A*: nodos expandidos. Greedy expande menos nodos (no busca optimalidad) pero A* garantiza costo optimo. A*(static_deadlock) expande mucho mas porque $h = 0$ no guia.

6.5. Distribucion de costo

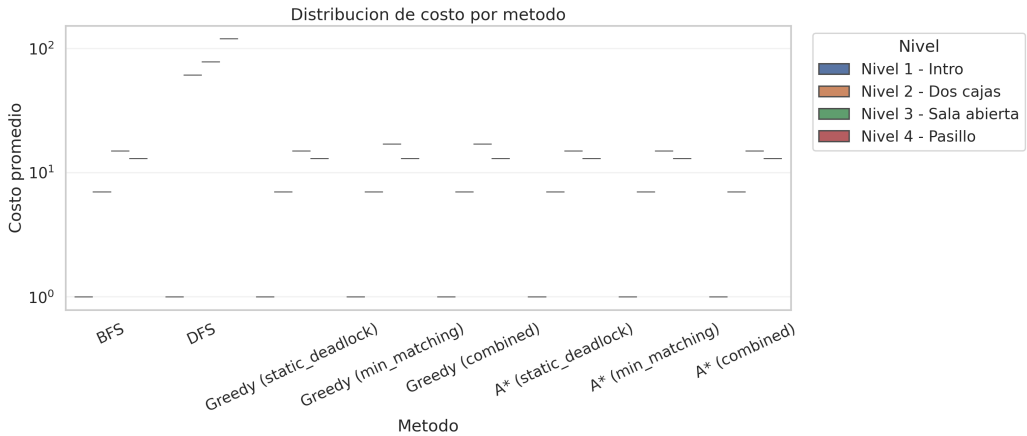

Figura 9: Distribucion de costo por metodo. Las cajas son planas porque la busqueda es determinista (los nodos expandidos, frontera y costo no varian entre iteraciones; solo el tiempo fluctua por ruido del OS).

7. Analisis

7.1. Optimalidad

BFS y todas las variantes de A* encuentran la solucion optima en todos los niveles. DFS encuentra soluciones hasta 9× mas largas que el optimo (120 vs 13 en Nivel 4). Greedy con min_matching empata el optimo en 3 de 4 niveles pero no lo garantiza (en Nivel 3 da costo 17 vs optimo 15).

7.2. Eficiencia de las heuristicas

La heuristica de **min_matching** es significativamente mas informativa que **static_deadlock**:

- A*(min_matching) expande 133 nodos en Nivel 3 vs 680 de A*(static_deadlock) — 5× menos.
- La razon: static_deadlock devuelve $h = 0$ para estados no muertos, funcionando como test de poda binario pero sin guiar la busqueda. min_matching da informacion numerica de cuanto falta.
- La combinada (combined) no aporta mejora sobre min_matching sola en estos niveles, porque min_matching ya es consistente y no hay reaperturas.

7.3. Rol de static_deadlock

Aunque static_deadlock aporta poco como heuristica numerica, su rol como **mecanismo de poda** es valioso: en niveles con posibilidades de deadlock, descarta ramas enteras devolviendo ∞. Esto explica por que A*(static_deadlock) expande menos que BFS pese a tener $h = 0$ para la mayoria de estados.

7.4. Determinismo de las iteraciones

La busqueda es completamente determinista (orden fijo de sucesores, semillas fijas). Las 10 iteraciones producen exactamente los mismos nodos expandidos, frontera y costo. Solo el tiempo de ejecucion varia por ruido del sistema operativo. Las barras de error en metricas distintas al tiempo son de ancho cero.

8. Metricas reportadas

Al finalizar cada corrida se reporta:

1. **Resultado**: exito o fracaso.
2. **Costo de la solucion**: cantidad de acciones del plan (cada movimiento del jugador cuesta 1).
3. **Nodos expandidos**: estados sacados de la frontera y cuyos sucesores fueron generados.
4. **Nodos frontera**: tamano de la frontera al momento de terminar la busqueda.
5. **Solucion**: secuencia de acciones (UP, DOWN, LEFT, RIGHT) desde el estado inicial al goal.
6. **Tiempo de procesamiento**: medido con `time.perf_counter()`.

Adicionalmente, para A*, se reportan metricas de diagnostico: `stale_skipped` (entradas obsoletas en el heap), `reopened_states` (nodos cerrados reabiertos) y `heuristic_cache_hits`.

9. Tests

El proyecto incluye 36 tests unitarios distribuidos en 7 archivos:

- **test_search_astar.py** (5 tests): reapertura de nodos, entradas stale, tie-breaking, cache de heuristica, politica de deadlocks.
- **test_search_algorithms.py** (6 tests): DFS encuentra solucion, Greedy encuentra solucion, Greedy $\geq$ BFS en costo, estado ya resuelto retorna costo 0, roundtrip de render, guard de recursion.
- **test_min_matching.py** (9 tests): cajas en goals, una caja, matching optimo vs greedy, admisibilidad, combinada, A* iguala BFS en costo.
- **test_static_deadlocks.py** (8 tests): goals no prohibidos, esquinas prohibidas, BFS inverso, poda de sucesores, heuristica infinito, combinada, cache.
- **test_level_io.py** (3 tests): tableros irregulares, validacion de entidades, carga de archivos.
- **test_deadlock_policy.py** (2 tests): flags CLI, forwarding de politica.
- **test_benchmark_script.py** (2 tests): parsing de specs, forwarding por entrada.
- **test_experiment_runner.py** (1 test): pipeline completo genera outputs esperados.

10. Conclusiones

1. **Las heuristicas informadas reducen drasticamente la exploracion.** A*(min_matching) expande 31× menos nodos que BFS en el nivel con mayor branching.
2. **No todas las heuristicas son iguales.** static_deadlock es una poda binaria (0 o ∞) util para descartar estados imposibles, pero no guia la busqueda. min_matching da informacion numerica que ordena eficientemente la frontera.
3. **La combinacion con max es segura.** $\text{máx}(h_1, h_2)$ preserva admisibilidad y aprovecha ambas senales. En estos niveles, combined no mejora sobre min_matching sola, pero en niveles mas grandes con deadlocks la poda seria mas valiosa.
4. **A* con reapertura garantiza optimalidad con heuristicas admisibles.** La implementacion reabre nodos cerrados cuando encuentra un camino mas barato, eliminando la necesidad de consistencia para la garantia de optimalidad.
5. **DFS no es adecuado para Sokoban.** Encuentra soluciones hasta 9× mas largas que el optimo, con el mayor numero de nodos expandidos.
6. **Greedy es un buen compromiso practico.** Con min_matching, empata o se acerca al optimo en estos niveles y explora una fraccion minima del espacio. Pero no garantiza optimalidad.